



SECURITY ASSESSMENT Butterfly Life Staking

September 30, 2026

Audit Status: Pass

BLADE POOL

RISK ANALYSIS | Butterfly Life.

■ Classifications of Manual Risk Results

Note: Risk classifications follow CVSS v4.0 standards (<https://www.first.org/cvss/v4.0/specification-document>). Contract security metrics are verified through GoPlus API token scanner technology, which provides real-time validation of contract parameters including

Classification	CVSS Range	Description
 Critical	9.0 - 10.0	Critical vulnerabilities that pose immediate and severe risks requiring urgent attention.
 High	7.0 - 8.9	High-priority issues that could lead to significant security breaches or financial loss.
 Medium	4.0 - 6.9	Medium-severity findings that should be addressed to improve contract security.
 Low	0.1 - 3.9	Low-risk items or best practice suggestions with minimal security impact.
 Informational	0.0	Informational findings about detected functions or contract features.

■ Manual Code Review Risk Results

Contract Security	Description
 Buy Tax	0% (staked token)
 Sale Tax	0% (staked token)
 Cannot Buy	N/A
 Cannot Sale	N/A
 Max Tax	N/A
 Modify Tax	N/A
 Fee Check	Not Applicable
 Is Honeypot?	Not Detected
 Trading Cooldown	Not Detected
 Enable Trade?	N/A

Contract Security	Description
 Pause Transfer?	Not Detected
 Max Tx?	Not Detected
 Is Anti Whale?	Not Detected
 Is Anti Bot?	Not Detected
 Is Blacklist?	Not Detected
 Blacklist Check	Not Detected
 Is Whitelist?	Not Detected
 Can Mint?	No mint capability
 Is Proxy?	Not Detected
 Can Take Ownership?	Standard OpenZeppelin Ownable; owner cannot withdraw staked principal
 Hidden Owner?	Not Detected
 Owner	0x217eD5cD4f3eC63bE345dD8Ea705E063B205Fdbf
 Self Destruct?	Not Detected
 External Call?	Present (staked token, AAPLB token, Vault, Card NFT)
 Other?	400-day linear release; AAPLB dividends; referral and card rewards
 Holders	N/A (staking contract)
 Audit Confidence	Good - Pass (preliminary; principal not owner-withdrawable)
 Authority Check	Owner-controlled
 Freeze Check	Not Detected

Overview of identified vulnerabilities and risks. See the full report for detailed methodology and recommendations.



Butterfly Life

Executive Summary

TYPES

DeFi

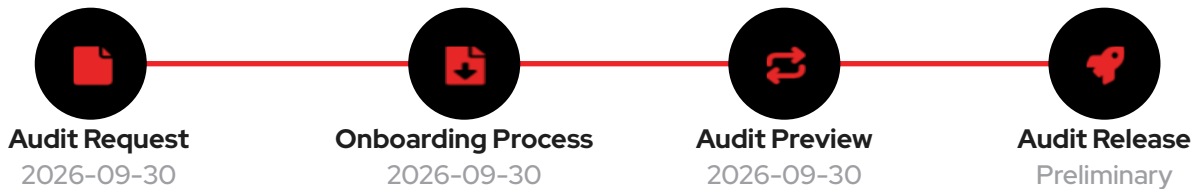
ECOSYSTEM

BNB Smart Chain

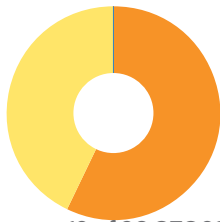
LANGUAGE

Solidity

Timeline



Vulnerability Summary



7
Total Findings

0
Resolved

7
Pending

7
Unresolved

19 of 26 CFG01-CFG26 checks passed with no security finding; 7 open findings. Slither 0.11.5 static analysis.

Severity	Resolution Status	Description
0 Critical		Immediate danger. Can lead to loss of user funds, unauthorized access, or full system compromise.
0 High		Significant risk that should be addressed urgently to prevent exploitation or financial loss.
4 Medium	0 Resolved, 4 Pending	Moderate risk that can lead to issues if combined with other weaknesses; should be addressed.
3 Low	0 Resolved, 3 Pending	Minor concern or best-practice deviation with limited direct security impact.
0 Informational		Not a vulnerability - code-quality, documentation, or optimization suggestions.

Industry Standards Compliance

This audit follows internationally recognized security standards to provide comprehensive assurance.

EEA EthTrust Security Levels

EthTrust/EVM review conducted for a BNB Smart Chain contract.

Smart Contract Security Verification Standard (SCSVS)

SCSVS review conducted for an EVM staking contract.

CVSS v3.1 Severity Scoring

Per-finding CVSS recorded on each CFG item.

PROJECT OVERVIEW | Butterfly Life.

Token Summary

Parameter	Result
Address	0x4D02155ED7912316b35a12bC5eB130302683d0fa
Name	Butterfly Life
Token Tracker	Butterfly Life (BUTTERFLY)
Decimals	18
Supply	1,000,000,000 (staked token)
Platform	BNB Smart Chain
Compiler	v0.8.29+commit.ab55807c
Contract Name	Stake
Optimization	No
LicenseType	MIT (SPDX)
Language	Solidity
Codebase	https://bscscan.com/ address/0x4D02155ED7912316b35a12bC5eB130302683d0fa#code

Main Contract Assessed

Name	Contract	Live
Butterfly Life	0x4D02155ED7912316b35a12bC5eB130302683d0fa	Yes
Butterfly Life token (staked)	0x87aE267002cD1eA0eC86Bf3601D4a8AD76A47777	Yes
AAPLB dividend token	0x431a3BEE82E2ca41e49895CbECE5bB0F76A89b7A	No
Butterfly card NFT	0x4788a300A96Cccdaf87A0C61e288CbFE14aE55EC	Yes
Vault (card rewards)	0xf867e9Ce44aCb81D3aB82e431B80942Bdf9A24B8	Yes

TestNet Contract Was Not Assessed

Solidity Code Provided

SolID	File Sha-1	FileName
Stake	d505dac232c445b25705fcc0b98a21ae15e89b9d	StakeFinal.sol
Stake	N/A	Vault.sol (linked, reviewed for interaction)
Stake	N/A	Card.sol (linked, reviewed for interaction)
Stake	N/A	N/A
Stake	N/A	N/A
Stake	N/A	N/A

TECHNICAL FINDINGS

Smart contract security audits classify risks into several categories: Critical, High, Medium, Low, and Informational. These classifications help assess the severity and potential impact of vulnerabilities found in smart contracts.

Classification of Risk

Severity	CVSS Range	Description
 Critical	9.0 - 10.0	Immediate danger. Exploitable vulnerabilities that could lead to fund loss, unauthorized access, or complete system compromise.
 High	7.0 - 8.9	Significant security risk. Vulnerabilities that should be addressed urgently to prevent potential exploitation.
 Medium	4.0 - 6.9	Moderate security risk. Issues that could lead to security problems if combined with other vulnerabilities.
 Low	0.1 - 3.9	Minor security concern. Best practice violations or low-impact issues.
 Informational	0.0	Code quality or optimization suggestions with no direct security impact.

Slither Static Analysis Results

Slither is a Solidity static analysis framework that runs a suite of vulnerability detectors to identify potential security issues, code quality problems, and best practice violations. It provides comprehensive security analysis with confidence ratings for each finding. For full detector documentation, visit: <https://github.com/crytic/slither/wiki/Detector-Documentation>

Impact Level	Count	Description
High	0	Critical security vulnerabilities requiring immediate attention (e.g., reentrancy, unchecked transfers)
Medium	7	Significant issues that should be addressed (e.g., dangerous comparisons, arithmetic issues)
Low	14	Minor issues and best practice violations (e.g., missing events, naming conventions)
Informational	54	Code quality and optimization suggestions (e.g., unused variables, gas optimizations)
TOTAL	75	Total findings across all severity levels

Analysis Summary

Slither (101 detectors, solc 0.8.29) reported 75 results: 0 High; 7 Medium (6 uninitialized-local and 1 strict-equality, all false positives: locals default to zero and totalStakeAmount == 0 is an intentional guard); 14 Low (11 timestamp comparisons for the 400-day linear release, 3 external calls in loops - see CFG04); 54 Informational/Optimization (the hardcoded addresses could be constant, which also confirms none of them can be changed).

Remediation Approach

All High and Medium impact findings from Slither have been manually reviewed and mapped to CFG test cases where applicable. Each finding has been analyzed in the context of the contract's business logic to determine actual exploitability. Detailed analysis of specific findings is available in the Technical Findings section of this audit report.

Deprecation Notice

The Smart Contract Weakness Classification Registry (SWC Registry) was deprecated in September 2023. CFGNinja audits now use Slither for static analysis, which provides more comprehensive and up-to-date vulnerability detection. SWC was an implementation of the weakness classification scheme proposed in EIP-1470, loosely aligned to the Common Weakness Enumeration (CWE). For historical reference: <https://swcregistry.io/>

Detailed Security Findings

Each finding lists its severity, CVSS, location and description, the affected source excerpt (verbatim from the verified Blockscout source), our recommendation, mitigation, references, and a suggested fix where the remediation is code-expressible.

BUTTERFLY-04 | Unbounded Loops in Claim Functions.

Category	Severity	CVSS	Location	Status
Denial-of-Service	 Low	2.5	StakeFinal.sol: Stake.cl aimButterflyTotal() / claimCardsReward() / getUserCardsInfo()	 Detected

Description

claimButterflyTotal() iterates every stake order the user has ever made, and claimCardsReward() / getUserCardsInfo() iterate every card ever recorded for the user with an external ownerOf() call per entry. Orders are never removed, so a heavy user can eventually exceed the block gas limit. Per-order (claimButterflyByOrder) and per-card (claimSingleCardReward) alternatives exist, so funds are not locked.

Recommendation

Add paginated variants (start, count), or remove fully released orders from iteration.

Mitigation

Mitigated by per-order and per-card claim functions.

References:

<https://swcregistry.io/docs/SWC-128>

BUTTERFLY-05 | Missing Events on Admin and Card Actions.

Category	Severity	CVSS	Location	Status
Logging	 Informational	0.0	StakeFinal.sol: setRewardToken() / claimCardsReward() / c laimSingleCardReward()/ activateCard()	 Detected

Description

setRewardToken(), card reward claims and card activation emit no events, which makes off-chain monitoring of these actions harder.

Recommendation

Emit events for reward-token changes, card activations and card reward claims.

Mitigation

Disclosure only.

References:

N/A

BUTTERFLY-08 | Card Holder Index Can Desynchronize From NFT Ownership.

Category	Severity	CVSS	Location	Status
Code Quality	 Low	2.0	StakeFinal.sol: Stake.on CardTransfer() / butter flyNFT._update()	 Detected

Description

userCards is updated only through the NFT transfer hook, which is skipped for transfers to or from the NFT owner or the staking contract, when the hook is disabled or skipped per token, and whenever onCardTransfer() reverts (the NFT swallows the error with try/catch). The list then no longer matches ownerOf(), so claimCardsReward() and getUserCardsInfo() can omit cards, and a later removeUserCard() on a stale entry reverts. Code quality notes: settle() uses 'if' instead of 'else if' for the first branch, commented-out code remains, and a 'test data, change for production' comment sits above the hardcoded addresses (verified: the deployed addresses are the live contracts).

Recommendation

Settle and update the index inside the hook for every transfer, or drop the index and read ownership directly. Remove dead code and stale comments.

Mitigation

Mitigated by claimSingleCardReward(), which checks ownerOf() directly.

References:

<https://swcregistry.io/docs/SWC-105>
<https://owasp.org/www-project-smart-contract-top-10/>

BUTTERFLY-13 | Dividend Accumulator Precision Loss Strands AAPLB.

Category	Severity	CVSS	Location	Status
Accounting / Precision	 Medium	4.3	StakeFinal.sol: Stake.updatePool()	 Detected

Description

updatePool() adds $(\text{newApple} * 1e12) / \text{totalStakeAmount}$ to accApplePerShare and then sets lastAppleBalance to the full current balance. With ~206M staked tokens ($2.06e26$ wei), any increment below $\text{totalStakeAmount} / 1e12$ (~0.000206 AAPLB today, up to 0.001 AAPLB at the 1B cap) rounds to zero, yet the balance is marked as distributed. Because updatePool() runs on every stake and claim, frequent small AAPLB inflows can be partly or fully truncated each time. The truncated AAPLB is never assigned to anyone and stays in the contract permanently. The dust cannot be quantified without indexing every position, but it grows with activity.

Recommendation

Raise ACC_PRECISION to $1e18$ or more, and only advance lastAppleBalance by the amount actually distributed (carry the remainder: $\text{lastAppleBalance} += \text{accIncrement} * \text{totalStakeAmount} / \text{ACC_PRECISION}$).

Mitigation

Open - pending client response.

References:

<https://swcregistry.io/docs/SWC-101>

BUTTERFLY-16 | Reward Token Unset and Freely Replaceable by Owner.

Category	Severity	CVSS	Location	Status
Centralization	 Medium	5.0	StakeFinal.sol: Stake.se tRewardToken() / claimRewardToken()	 Detected

Description

Each stake credits pendingRewardAmount (4% of the first 200M staked, stepping down to 0.25% up to 1B). By historyStakeAmount this is about 8.12M reward tokens already owed. At block 124,916,212 rewardToken is address(0), so every claimRewardToken() call reverts. The owner alone decides which token is paid out, can change it at any time without a timelock or event, and must separately fund the contract. Stakers' accrued allocation therefore depends entirely on owner action and could be paid in any token the owner chooses.

Recommendation

Set the reward token once (immutable or one-time setter), emit an event, fund it before accepting claims, and publish the intended token address to stakers.

Mitigation

Open - reward token not configured at time of review.

References:

<https://swcregistry.io/docs/SWC-105>

<https://owasp.org/www-project-smart-contract-top-10/>

BUTTERFLY-18 | Economic Design Disclosure.

Category	Severity	CVSS	Location	Status
Transparency	 Informational	0.0	StakeFinal.sol: Stake (economics)	 Detected

Description

Principal cannot be withdrawn early: it is released linearly at 0.25% per day and fully available after 400 days. Of each AAPLB dividend claim, the staker receives 50%, 10% goes to poolAddress (an EOA), 15% to up to 5 referral uplines by level (remainder to balanceAddress, an EOA), and 25% to the card Vault. The staking yield therefore depends on AAPLB arriving from outside the contract and on the owner funding the reward token. The contract does not generate yield itself.

Recommendation

Publish these parameters clearly to stakers before they deposit.

Mitigation

Disclosure only.

References:

N/A

BUTTERFLY-20 | Owner Controls the Linked Vault and Card NFT.

Category	Severity	CVSS	Location	Status
Centralization / Ownership	 Medium	5.0	StakeFinal.sol: Vault.setStakeSystem() / butterflyNFT.setStakeSystem(), toggleHook(), setSkipHookToken()	 Detected

Description

The staking contract's owner (0x217e...Fdbf) also owns the Vault that holds the 25% card-holder share of all dividends and the Card NFT contract. Vault.setStakeSystem() can repoint stakeAddress to any address, which could then call withdrawToUser() and drain the Vault. On the NFT, the owner can switch the transfer hook off globally or per token. The owner is a single EOA using an EIP-7702 delegation, not a multisig or timelock. Staked principal itself is NOT at risk: the staking contract has no owner withdrawal path.

Recommendation

Make Vault.stakeAddress immutable (or one-time set), move ownership of Stake, Vault and Card to a multisig with a timelock, and document the privileged roles.

Mitigation

Open - pending client response.

References:

<https://swcregistry.io/docs/SWC-105>

<https://owasp.org/www-project-smart-contract-top-10/>

BUTTERFLY-22 | Referral Team Performance Is Never Reduced (Level Farming).

Category	Severity	CVSS	Location	Status
Manipulation Prevention	 Medium	4.3	StakeFinal.sol: Stake.stakeButterfly() / getUserBankLevel() / setTeamReward()	 Detected

Description

Each stake adds its amount to teamPerform of up to 5 upline addresses, and getUserBankLevel() derives a permanent level (3M/10M/30M/60M/100M tokens) from that figure. teamPerform is never reduced when principal is released and withdrawn. A single actor can stake through a second wallet referred by their own first wallet, reach a level, withdraw the principal over 400 days, and keep the level forever. Levels decide who captures the 15% team share of every downline AAPLB claim (20% to 100% of it), which otherwise goes to balanceAddress. Self-referral cannot be prevented on-chain, but the lack of any decrement makes the gamed level permanent.

Recommendation

Decrease teamPerform along the upline when principal is claimed (mirror the stake loop), or base levels on current effective stake rather than historical volume.

Mitigation

Open - pending client response.

References:

<https://consensys.github.io/smart-contract-best-practices/>
<https://swcregistry.io/docs/SWC-105>
<https://owasp.org/www-project-smart-contract-top-10/>

BUTTERFLY-23 | Anonymous Team - No KYC Verification.

Category	Severity	CVSS	Location	Status
Team Transparency	 Informational	0.0	StakeFinal.sol: Project team	 Detected

Description

No team member names, credentials or verifiable identities are disclosed, no website or social channels were provided for this review, and no KYC has been completed with a recognized provider. This is flagged for transparency and caps the audit score at 90 under CFG Ninja policy. It does not, by itself, indicate a code vulnerability.

Recommendation

Complete KYC with a recognized provider and publish official project channels.

Mitigation

No KYC or team disclosure found.

References:

N/A

BUTTERFLY-24 | No Recovery Path for Stranded Tokens.

Category	Severity	CVSS	Location	Status
Input Validation	 Low	2.0	StakeFinal.sol: Stake.updatePool() / constructor	 Detected

Description

AAPLB received while `totalStakeAmount == 0` is absorbed into `lastAppleBalance` and never distributed. The same applies to rounding dust from the 50/10/15/25 split and the precision loss in CFG13. The constructor also gives `initialParent` a phantom 1-wei stake that is not counted in `totalStakeAmount`. The contract has no rescue function, so any stranded AAPLB or unrelated tokens sent to it are locked permanently.

Recommendation

Add an owner-restricted sweep limited to the undistributed AAPLB surplus and to tokens other than the staked token.

Mitigation

Open - pending client response.

References:

<https://swcregistry.io/docs/SWC-105>

<https://owasp.org/www-project-smart-contract-top-10/>

FINDINGS

In this document, we present the findings and results of the smart contract security audit. The identified vulnerabilities, weaknesses, and potential risks are outlined, along with recommendations for mitigating these issues. It is crucial for the team to address these findings promptly to enhance the security and trustworthiness of the smart contract code.

Severity	Found	Pending	Resolved
 Critical	0	0	0
 High	0	0	0
 Medium	4	4	0
 Low	3	3	0
 Informational	0	0	0
Total	7	7	0

In a smart contract, a technical finding summary refers to a compilation of identified issues or vulnerabilities discovered during a security audit. These findings can range from coding errors and logical flaws to potential security risks. It is crucial for the project owner to thoroughly review each identified item and take necessary actions to resolve them. By carefully examining the technical finding summary, the project owner can gain insights into the weaknesses or potential threats present in the smart contract. They should prioritize addressing these issues promptly to mitigate any risks associated with the contract's security. Neglecting to address any identified item in the security audit can expose the smart contract to significant risks. Unresolved vulnerabilities can be exploited by malicious actors, potentially leading to financial losses, data breaches, or other detrimental consequences. To ensure the integrity and security of the smart contract, the project owner should engage in a comprehensive review process. This involves understanding the nature and severity of each identified item, consulting with experts if needed, and implementing appropriate fixes or enhancements. Regularly updating and maintaining the smart contract's codebase is also essential to address any emerging security concerns. By diligently reviewing and resolving all identified items in the technical finding summary, the project owner can significantly reduce the risks associated with the smart contract and enhance its overall security posture.

SOCIAL MEDIA CHECKS | Butterfly Life.

Social Media		URL	Result
Website			N/A
Telegram			N/A
Twitter			N/A
Facebook			N/A
Reddit	N/A		N/A
Instagram	N/A		N/A
CoinGecko			N/A
Github	N/A		N/A
CMC			N/A
Email			Contact
Other			N/A

From a security assessment standpoint, inspecting a project's social media presence is essential. It enables the evaluation of the project's reputation, credibility, and trustworthiness within the community. By analyzing the content shared, engagement levels, and the response to any security-related incidents, one can assess the project's commitment to security practices and its ability to handle potential threats.

Social Media Information Notes:

Auditor Notes: No website or social channels were provided for this review. No CoinGecko/CMC listing was checked. Team is anonymous; no KYC.

Project Owner Notes: No socials provided.

Assessment Results

Final Audit Score BUTTERFLY.

Review	Score
Security Score	85
Auditor Score	85

Our security assessment or audit score system for the smart contract and project follows a comprehensive evaluation process to ensure the highest level of security. The system assigns a score based on various security parameters and benchmarks, with a passing score set at 80 out of a total attainable score of 100. The assessment process includes a thorough review of the smart contracts codebase, architecture, and design principles. It examines potential vulnerabilities, such as code bugs, logical flaws, and potential attack vectors. The evaluation also considers the adherence to best practices and industry standards for secure coding. Additionally, the system assesses the projects overall security measures, including infrastructure security, data protection, and access controls. It evaluates the implementation of encryption, authentication mechanisms, and secure communication protocols. To achieve a passing score, the smart contract and project must attain a minimum of 80 points out of the total attainable score of 100. This ensures that the system has undergone a rigorous security assessment and meets the required standards for secure operation.



Important Notes for BUTTERFLY

Butterfly Life - Stake - Smart Contract Security Audit Report (PRELIMINARY)

PROJECT OVERVIEW

Butterfly Life is a staking system on BNB Smart Chain. Users stake the Butterfly Life token (1,000,000,000 supply, 0% tax, ownership renounced) with a referrer. Principal is released linearly at 0.25% per day over 400 days. Stakers earn a pro-rata share of AAPLB tokens that arrive in the contract, a one-time allocation of a separate reward token on early staking volume, and referral rewards through a 5-level hierarchy. 25% of every dividend claim funds a Vault that pays holders of 668 Butterfly card NFTs.

This is a PRELIMINARY report for client review. Findings are open pending the client's response.

CONTRACT DETAILS

Name: Stake (StakeFinal.sol)

Address: 0x4D02155ED7912316b35a12bC5eB130302683d0fa

Chain: BNB Smart Chain (chainId 56)

Compiler: Solidity v0.8.29+commit.ab55807c | Optimizer: 2000 runs | License: MIT (SPDX)

Verification: Sourcify exact match

Owner / Deployer: 0x217eD5cD4f3eC63bE345dD8Ea705E063B205Fdbf (EOA with EIP-7702 delegation)

Deployed: 2026-09-28

Staked token: 0x87aE267002cD1eA0eC86Bf3601D4a8AD76A47777

Linked: AAPLB 0x431a...9b7A, Card NFT 0x4788...55EC, Vault 0xf867...24B8

ON-CHAIN STATE (block 124,916,212)

Total staked: ~205.98M tokens (about 20.6% of supply); contract token balance matches exactly.

Reward token: NOT SET (address zero) - reward-token claims currently revert.

Vault and Card NFT both point to this staking contract; all 668 cards are minted.

SECURITY FINDINGS

Dividend Accumulator Precision Loss Strands AAPLB (MEDIUM - CFG13)

updatePool() uses 1e12 precision against ~2.06e26 wei staked, so increments below ~0.000206 AAPLB round to zero while the balance is still marked as distributed. The truncated AAPLB stays in the contract permanently.

Recommendation: Raise precision to 1e18+ and carry the undistributed remainder.

Referral Team Performance Is Never Reduced (MEDIUM - CFG22)

teamPerform only increases. Levels (which capture the 15% team share of downline claims) stay permanent after principal is withdrawn and can be farmed through self-referral.

Recommendation: Decrease teamPerform on principal claims or base levels on current stake.

Reward Token Unset and Freely Replaceable by Owner (MEDIUM - CFG16)

About 8.12M reward tokens are already owed, but rewardToken is unset and the owner can set or change it at any time without an event or timelock.

Recommendation: Set the reward token once, emit an event and fund it before claims.

Owner Controls the Linked Vault and Card NFT (MEDIUM - CFG20)

The same owner can repoint the Vault's stakeAddress and drain the 25% card-holder share, and can disable the NFT transfer hook. Staked principal is not at risk.

Recommendation: Make Vault.stakeAddress immutable and move ownership to a multisig with timelock.

Unbounded Loops in Claim Functions (LOW - CFG04)

claimButterflyTotal() and claimCardsReward() iterate all historical orders and cards. Per-order and per-card claims exist as fallbacks.

Card Holder Index Can Desynchronize From NFT Ownership (LOW - CFG08)

userCards is updated only through a hook that can be skipped or silently fail. claimSingleCardReward() remains correct.

No Recovery Path for Stranded Tokens (LOW - CFG24)

AAPLB received while nothing is staked, rounding dust and mistaken transfers are locked permanently.

Economic Design Disclosure (INFORMATIONAL - CFG18)

400-day linear principal release; dividend split 50% staker / 10% pool / 15% referral / 25% card Vault. Yield depends on outside AAPLB inflows.

Missing Events (INFORMATIONAL - CFG05) and Anonymous Team - No KYC (INFORMATIONAL - CFG23)

POSITIVE FINDINGS

Principal Safety: the owner has no function to withdraw staked tokens.

Reentrancy: every user entry point is nonReentrant; SafeERC20 throughout.

Immutable Wiring: token, Vault and Card addresses have no setters.

Card Funding: at full card supply the per-card rates equal exactly the 25% Vault share.

Staked Token: 0% tax, not mintable, ownership renounced (GoPlus).

AUDIT METHODOLOGY

Manual line-by-line review of the verified Sourcify source against the CFG01-CFG26 framework, Slither static analysis (75 results triaged; no High), on-chain state reads over BSC RPC and GoPlus token and address scans. The linked Vault and Card NFT sources were reviewed for their interaction with the staking contract.

Audit Date: September 30, 2026 | Chain: BNB Smart Chain | Auditor: CFG.NINJA | Status: PRELIMINARY

DISCLAIMER

This audit does not guarantee the absence of all vulnerabilities. It covers the Stake contract; the AAPLB token was out of scope. Users should conduct their own research before interacting with any smart contract.

I Appendix

Finding Categories

Centralization / Privilege

Centralization / Privilege findings refer to either feature logic or implementation of components that act against the nature of decentralization, such as explicit ownership or specialized access roles in combination with a mechanism to relocate funds.

Gas Optimization

Gas Optimization findings do not affect the functionality of the code but generate different, more optimal EVM opcodes resulting in a reduction in the total gas cost of a transaction.

Logical Issue

Logical Issue findings detail a fault in the logic of the linked code, such as an incorrect notion of how `block.timestamp` works.

Control Flow

Control Flow findings concern the access control imposed on functions, such as owner-only functions being invocable by anyone under certain circumstances.

Volatile Code

Volatile Code findings refer to segments of code that behave unexpectedly in certain edge cases that may result in a vulnerability.

Coding Style

Coding Style findings usually do not affect the generated byte-code but rather comment on how to make the codebase more legible and, as a result, easily maintainable.

Inconsistency

Inconsistency findings refer to functions that should seemingly behave similarly yet contain different code, such as a constructor assignment imposing different require statements on the input variables than a setter function.

Coding Best Practices

ERC 20 Coding Standards are a set of rules that each developer should follow to ensure the code meets a set of criteria and is readable by all developers.

Disclaimer

Scope. This report documents the results of a security assessment and smart contract audit performed by Bladepool / CFG NINJA (the "Auditor") for the project specified in this report. The assessment covers only the deliverables and code expressly listed in the engagement; any changes, integrations, or subsequent deployments are outside the scope.

No Warranty. The Auditor performs the assessment using industry-standard practices but makes no representations or warranties, express or implied, including any warranty of merchantability, fitness for a particular purpose, or non-infringement. The Auditor does not guarantee that the audited code is free of bugs, vulnerabilities, or exploitable issues.

Limitation of Liability. To the maximum extent permitted by law, the Auditor and its affiliates, officers, employees, agents, and contractors shall not be liable for any indirect, incidental, special, consequential, or punitive damages, or for any loss of profits, revenue, data, or business opportunities, arising out of or related to the assessment, even if advised of the possibility of such damages. The Auditor's aggregate liability for direct damages is limited to the fees paid for the audit engagement.

Client Responsibility. The project owner/client retains sole responsibility for all decisions and actions taken in connection with the audited code, including fixing identified issues, deploying code to any environment, and applying security mitigations. The Auditor's recommendations are advisory; implementation and verification of fixes are the client's responsibility.

Third-Party Tools and Services. The Auditor may use third-party tools, services, or automated scanners as part of the assessment. Use of such tools is without warranty; the Auditor is not responsible for defects, failures, or inaccuracies arising from third-party services.

Confidentiality and Data Security. The Auditor will treat non-public client information as confidential. However, the Auditor cannot guarantee absolute security for data transmitted over the internet or third-party platforms. Client should take reasonable precautions to protect sensitive information.

Not Financial, Legal, or Investment Advice. The assessment is a technical security review only and is not financial, legal, tax, or investment advice. Clients should consult appropriate professionals for those matters.

Ownership and Governance Notes. If ownership is transferred, held by a multisig, or controlled by a governance contract, the client should document and disclose those arrangements; the Auditor's report reflects the ownership state observed during the engagement but may not reflect subsequent changes.

Governing Law and Counsel. This engagement and any disputes arising from it shall be governed by the laws agreed in the engagement terms. Clients are advised to seek legal counsel before relying on or publishing the report.

Acceptance. By proceeding with this engagement or using this report, the client acknowledges and accepts these terms.

